

TRACKED

Tracked findings — §11.4.261(D)

Every finding the §11.4.261 audit sweep emits must be either **CLOSED** (the code genuinely repaired, with captured evidence, so the finding disappears from the sweep) or **TRACKED** here, with a real item, an honest provenance marker and a reversible-safe mitigation. §11.4.261(D) names the third option — “silent absorption” — and forbids it at PASS-bluff severity. This file is that honesty seam: it is what makes the word "status":"tracked" in zero_findings_ledger.jsonl mean something.

A tracked item is not a closed one. Nothing in this file is a claim that a finding has been fixed. Each entry says what was measured, why it is still there, and what stops it causing harm before it is closed.

How this file is read by the machine. `scripts/audit/zero_findings_sweep.sh` looks for a heading of the exact form `## ZF-<class>` and, inside it, a line beginning `Mitigation:.` A finding whose class has no such entry is written to the ledger as `status: open`, which correctly turns `CM-EVERY-FINDING-CLOSED-OR-TRACKED` red. The sweep never invents a disposition to buy itself a green — a class that acquires its first finding reddens two gates until a human writes an entry here.

On Target: dates. The sweep’s unresolved detector reports any item below whose `Target: date` has passed. **No entry currently carries one.** That is deliberate and it is a limitation, not a clean bill: setting a completion window is an operator commitment, and this file will not manufacture one. The detector states the same limit on every run — *“an item with no Target: date cannot be past its window and is not reported”*. Add a `Target: YYYY-MM-DD` line to any entry below and the class becomes live from that moment.

ZF-gaps

Measured: 7 findings, every one a row in docs/constitution-adoption/INVENTORY.md whose recorded status is not CLOSED — G4, G5, G8, G9, G10, G11, G12. The sweep does not judge these; it reads the register this repository already maintains, so each finding carries a per-finding `tracker_ref` pointing at its own inventory row rather than at this entry.

Why still open. Three different reasons, and collapsing them would be the dishonest move:

- **G4** (§11.4.156 CI surface) and **G5** (git hooks as a travelling artefact) are PARTIAL because the remaining work is *outside this tree*. G4's residue is a provider-side setting changeable only in a provider UI, and an active deploy workflow in the `milosvasic.ru` submodule kept deliberately for production uptime. G5's residue is `.git/hooks/`, which git does not track.
- **G8** (§11.4.65 export mandate) and **G12** (no `PreToolUse` guard wired) are OPEN with no work in flight. These are the two entries here that a session could actually close.
- **G9, G10 and G11** are recorded as NO VERIFIED CURRENT STATUS. They are findings *because* nobody can currently say whether they are open or closed — an unknown is not a zero (§11.4.6). Re-auditing them is the work.

Mitigation: the register itself is the mitigation — each row states its own evidence and its own boundary, and `scripts/continuation-check.sh` cross-checks the gap statuses in `INVENTORY.md` against the four root carriers, so a status that drifts between the two is caught rather than quietly diverging.

ZF-weak-spots

Measured: 1 finding — `scripts/install_skills.sh` carries no `set -u` unset-variable guard. Every other tracked shell script under `scripts/` and `tests/` has one.

Why still open. That file is already registered as an **exemption** in `scripts/check-registry.tsv` with the reason "*installer: shells out to `npx skills add` to populate global skills; asserts nothing about this repository*". It is not a gate and it produces no verdict, so an unset variable there cannot turn a red instrument green. It is left visible here rather than added to a detector allow-list, because an allow-list entry hides the row from the next reader while this entry shows it to them.

Mitigation: the file asserts nothing about this tree, so its failure mode is a broken install run — loud, immediate, and reversible by re-running it. It touches no repository state and no gate verdict.

ZF-danger-zones

Measured: 9 findings across 8 files — three eval "\$_paths_output" in the vendored .specify/scripts/bash/ spec-kit helpers, three eval "\$var" in scripts/continuation-check.sh, scripts/rollback-agents-wizard.sh and scripts/setup-agents-wizard.sh, two git commit --no-verify inside scripts/verify-manifest-pins.sh, and one rm -rf "\$T" inside a printf template in scripts/verify-check-registry.sh.

Why still open. These are matches of a genuinely dangerous *pattern*, and whether a given one is dangerous *in context* is a judgement. They are kept in the ledger, visible, rather than filtered out by a narrowed detector — narrowing a detector to make a count fall is exactly the §11.4.261(E) fabrication:

- The two --no-verify commits in verify-manifest-pins.sh run **inside a mktemp -d throwaway git repository** built by that script's own mutation proof. Bypassing hooks there is the point: the sandbox has no hooks and the proof must not depend on the developer's.
- The rm -rf "\$T" in verify-check-registry.sh is **text inside a printf format string** that the mutation proof writes into a synthetic gate. It is not executed by the gate itself.
- The three eval "\$var" in this repository's own scripts expand a variable the same script assembled from its own literals, never from input.
- The three in .specify/scripts/bash/ are **vendored third-party code** this repository consumes and does not own. They cannot be fixed from here.

Mitigation: every instance in this repository's own scripts operates on a mktemp -d sandbox or on a locally-assembled literal; none reads external input, and none runs on the real tree. The vendored spec-kit copies are third-party and out of this repository's control — recorded rather than silently omitted, per §11.4.156(C)/§11.4.29. The ratchet ceiling is the live guard: this count cannot rise without the sweep refusing the seam.

ZF-skipped-tests

Measured: 9 findings — the `testIgnore` line and its explanatory comment in `_tests/playwright.config.js`, four `test.skip(...)` call sites across `_tests/tests/`, two `t.Skip(...)` in `_tools/containers/cmd/runtime-probe/main_test.go`, and the tracked `.github/workflows/ci.yml.disabled`.

Why still open. Each is a documented decision this repository can defend, and each is genuinely a skip:

- **ci.yml.disabled** is the §11.4.156(B) compliance decision recorded in `docs/constitution-adoption/DECISION-11-4-156-COMPLY.md`. Enforcement moved to a local pre-push hook. Re-enabling it would be the violation.
- **testIgnore** defers three live-site specs out of gate 6 and into `_tools/deploy-langs.sh`, which runs them against what was actually shipped. The measured cause was runner reachability — 77 sixty-second `page.goto` timeouts and **zero** genuine assertion failures — while `curl` reached both live sites with `http=200`. The specs run; they run *later*.
- The four `test.skip(...)` are **runtime-conditional**: three skip a non-chromium project, one skips a viewport where the control under test does not exist. They are guards against asserting something meaningless, not disabled assertions.
- The two `t.Skip(...)` fire when a Go toolchain or a container runtime is absent from the host. One of them already labels itself a documented skip.

Mitigation: none of these is a silently disabled assertion. The three deferred specs run at deploy time against the live sites — `LIVE_SPECS` in `_tools/deploy-langs.sh` names all four, verified present at HEAD — so the coverage moved rather than vanished. The conditional skips fail loudly if their condition is met and the assertion would have been meaningful. The ratchet ceiling stops a tenth skip being added without a decision.

Classes with no tracker entry

`shortcomings`, `todo-fixme`, `bluffs`, `divergent-stale-orphan` and uncatalogued measured **0** findings, so no entry exists for them and none should be written pre-emptively. This is deliberate: a class with no entry has an implicit ratchet ceiling of 0 **and** no disposition, so its first finding lands in the ledger as `status: open` and reddens both `CM-ZERO-FINDINGS-MONOTONE-RATCHET` and `CM-EVERY-FINDING-CLOSED-OR-TRACKED` at once. Writing a speculative entry now would pre-authorise a finding nobody has seen.

A 0 from a detector is not an empty class. Each detector prints its own recall limit on every run. uncatalogued in particular is the §11.4.118 discovery-pressure class: a genuinely uncatalogued anti-pattern is by definition one nobody has named, and no static detector can enumerate it. Its 0 is the weakest number the sweep prints, and the sweep says so itself.